

L10a: Online Learning

Regret, weighted majority, and multiplicative weights

CHEME 5800 Cornell University

Copyright Jeffrey Varner 2026. All Rights Reserved

Objectives for Today

How can we learn from a sequence of outcomes chosen by an adversary, and how do we judge how well we learned?

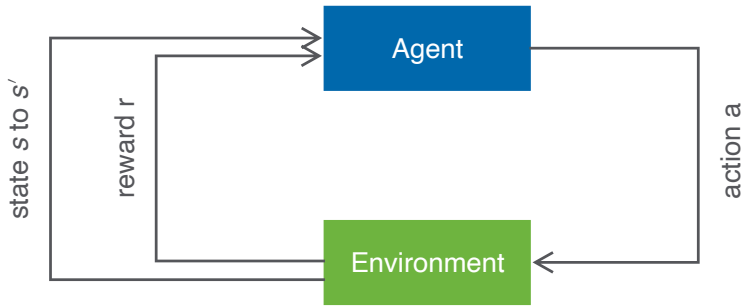
Today's concepts

- Describe the online learning problem: set it up as a game played over many rounds, in which we predict, an adversary reveals the outcome, and we learn from the result.
- Measure an online learning algorithm with regret: compare its loss with the best fixed strategy in hindsight, and see why we want a regret bound that grows more slowly than the number of rounds.
- Learn from expert advice with multiplicative weights: shrink the weight of each expert that does poorly, and bound how far we fall behind the best expert.

Lecture: Online Learning: Weighted Majority and Multiplicative Weights

The Reinforcement and Online Learning Problem

An agent observes a state $s \in \mathcal{S}$, takes an action $a \in \mathcal{A}$, and receives a reward r. How does it learn to choose actions that maximize its cumulative reward?



The agent observes its current state, selects an action, and receives a reward that influences its future decisions.

Three Families of Methods

- Multiplicative weights update expert weights based on their past performance.
- Bandit algorithms observe only the reward of the action they take; in the basic multi-armed bandit, there is no evolving state.
- Q-learning estimates the long-term value of each state-action pair, for environments with sequential dynamics.

The feedback determines how we can learn. With expert advice, the revealed outcome lets us score every expert. With bandit feedback, we see only the reward of our own action. In Q-learning, actions also change the state.

We begin with full-feedback expert advice.

Online Learning and Regret

For $t = 1, 2, \ldots, T$ **do**:

1. An *adversary* picks a number $y_t \in \{1, 2, \ldots, N\}$ and keeps it secret.
2. The *aggregator* (us) predicts $\hat{y}_t$, based on some strategy.
3. The adversary reveals y_t, and we incur the loss $l_t(\hat{y}_t, y_t)$.

The regret compares our total loss with the best fixed strategy in hindsight:

$$R(T) = \underbrace{\sum_{t=1}^{T} l_t(\hat{y}_t, y_t)}_{\text{aggregator}} - \underbrace{\min_u \sum_{t=1}^{T} l_t(u, y_t)}_{\text{best fixed strategy in hindsight}},$$

where the minimum is over every fixed prediction u. We want algorithms that perform *nearly* as well as the best fixed strategy.

Online Versus Supervised Learning

- Adaptation: online algorithms update their predictions one round at a time, and their guarantees do not assume the data come from a fixed distribution. Supervised algorithms are trained once on a fixed dataset.
- Regret minimization: regret depends on the algorithm and on the sequence of outcomes. A regret bound holds for every sequence, so it is a property of the algorithm.

With expert advice, the best fixed strategy is the best single expert: the expert whose predictions, which can change from round to round, had the smallest total loss.

Weighted Majority Algorithm

Littlestone and Warmuth (1994), in a prediction from expert advice problem:

- Game: each morning, we predict whether a stock goes up or down, encoded as $\{1, -1\}$. A wrong prediction costs a dollar. We watch the predictions of n experts, who may be correlated and may be wrong.
- Goal: bound our mistakes by a multiple of the best expert's mistakes plus an initial term, without knowing which expert is best.

Each expert starts with weight $w_i^{(1)} = 1$. In each round, we make a binary prediction $\hat{y}_t \in \{-1, 1\}$ and the adversary reveals y_t; if the weighted vote is tied, we predict -1.

Weighted Majority Algorithm: Pseudocode

For each round $t = 1, 2, \ldots, T$ **do**:

1. The aggregator predicts with the weighted majority of the experts: $\hat{y}_t = 1$ if the total weight of the experts predicting 1 is greater than the total weight of those predicting -1; otherwise, $\hat{y}_t = -1$.
2. The adversary reveals the actual outcome $y_t \in \{-1, 1\}$.
3. For each expert i who predicted incorrectly, update the weight:

   $$w_i^{(t+1)} = w_i^{(t)}(1 - \epsilon),$$

   where $0 < \epsilon \leq 1/2$ is a learning rate parameter.

Experts that predict well keep their weight, so their share of the vote grows.

Worked Example: Four Rounds

Three experts and $\epsilon = 1/2$. A prediction that differs from the outcome is a mistake, and each mistake halves that expert's weight:

Round	Weights before	Expert predictions	Our prediction	Outcome
1	$(1, 1, 1)$	$(1, 1, -1)$	1	1
2	$(1, 1, 0.5)$	$(-1, 1, 1)$	1	-1
3	$(1, 0.5, 0.25)$	$(-1, 1, 1)$	-1	1
4	$(0.5, 0.5, 0.25)$	$(1, -1, 1)$	1	1

In round 2, the experts predicting up carry weight 1.5 against 1, so we predict up; the outcome is down, so we make a mistake. In round 3, those two experts carry only 0.75, so we predict down.

Worked Example: The Weighted Vote



Expert 1 made one mistake and experts 2 and 3 made two. We made two, so our regret against the best expert is $R_{\text{WMA}}(4) = 2 - \min\{1, 2, 2\} = 1$.

Mistake Guarantee

Let $M_i^{(t)}$ be the mistakes of expert i up to time t, and $M^{(t)}$ our mistakes. For every expert i:

$$M^{(t)} \leq \frac{2 \ln n}{\epsilon} + 2(1+\epsilon)\, M_i^{(t)}$$

- The guarantee is multiplicative: our mistakes are at most $2(1+\epsilon)$ times the best expert's, plus an initial term that grows with the logarithm of the number of experts.
- The algorithm is deterministic: the same predictions and outcomes always produce the same votes.

Choosing an expert at random instead gives an additive guarantee.

Multiplicative Weights: Problem Setting

We follow one expert chosen at random in proportion to its weight. The adversary fixes the advice and the outcomes in advance.

- Sample an expert $i \sim \texttt{Categorical}(\mathbf{p}^{(t)})$ and follow its recommendation.
- Each expert's cost is $m_i^{(t)} \in [-1, 1]$: -1 if correct and $+1$ if incorrect.

The total expected cost and the regret against the best expert in hindsight are:

$$L_A = \sum_{t=1}^{T} \sum_{i=1}^{n} p_i^{(t)} m_i^{(t)}, \qquad R(T) = L_A - \min_i \sum_{t=1}^{T} m_i^{(t)}.$$

A sublinear regret bound grows more slowly than T, so the bound on $R(T)/T$ vanishes as $T \to \infty$.

Multiplicative Weights: Pseudocode

Initialize: Fix a learning rate $0 < \eta \leq 1/2$, and set $w_i^{(1)} = 1$ for each expert.

For $t = 1, 2, \ldots, T$ **do**:

1. Ask each expert i for its answer $\hat{y}_i^{(t)}$. Choose expert i with probability $p_i^{(t)} = w_i^{(t)} / \sum_{j=1}^{n} w_j^{(t)}$ and follow its answer.
2. The adversary reveals y_t. Compute each expert's cost: $m_i^{(t)} = -1$ if $\hat{y}_i^{(t)} = y_t$, and $+1$ otherwise.
3. Update the weight of each expert i:

$$w_i^{(t+1)} = w_i^{(t)} \cdot (1 - \eta \cdot m_i^{(t)}).$$

The learning rate η controls how quickly the weights shift toward the better experts.

Regret Bound

For costs $m_i^{(t)} \in [-1, 1]$ and $0 < \eta \le 1/2$, for any expert i:

$$\sum_{t=1}^{T} \mathbf{p}^{(t)} \cdot \mathbf{m}^{(t)} \le \sum_{t=1}^{T} m_i^{(t)} + \eta \sum_{t=1}^{T} |m_i^{(t)}| + \frac{\ln n}{\eta} \quad \Longrightarrow \quad R(T) \le \eta T + \frac{\ln n}{\eta},$$

taking i to be the best expert and using $|m_i^{(t)}| \le 1$.

Choosing $\eta = \sqrt{\ln n / T}$, which satisfies $\eta \le 1/2$ when $T \ge 4 \ln n$:

$$\boxed{R(T) \le 2\sqrt{T \ln n}}$$

The average regret per round is at most $2\sqrt{\ln n / T}$, which vanishes as $T \to \infty$: MWA does nearly as well as the best expert.

Compare the Two Guarantees

Both algorithms use full feedback: we score and update every expert.

Algorithm	Prediction rule	Guarantee
Weighted majority	Weighted binary vote	Mistakes at most $2(1+\epsilon)$ times the best expert's, plus $2\ln n/\epsilon$
Multiplicative weights	Sample an expert by weight	Expected cost at most $2\sqrt{T\ln n}$ above the best expert's

With $\ell_i^{(t)} = 1$ for a mistake and 0 otherwise, the signed cost is $m_i^{(t)} = 2\ell_i^{(t)} - 1$. The signed-cost regret is twice the expected mistake regret, so in mistakes the bound is $\sqrt{T\ln n}$.

Lab: Weighted Majority on the Stock Market

In the lab, we implement the weighted majority algorithm and use it to combine the advice of several experts:

- each expert is a ticker, such as QQQ or AAPL, predicting whether the SPY exchange-traded fund goes up or down each trading day,
- we check the mistake bound, first when the experts see the same day's price moves and then when they must forecast the next day.

Lab: The weighted majority algorithm on the stock market

Summary

We introduced online learning and two algorithms that learn from expert advice.

Key takeaways

- **Online learning and regret.** We set up online learning as a repeated game and measured an algorithm by its regret against the best fixed strategy in hindsight.
- **The weighted majority algorithm.** We took a weighted vote and shrank the weight of each expert that erred, which bounds our mistakes by two to three times the best expert's.
- **The multiplicative weights algorithm.** We followed an expert drawn by weight and updated every weight by its cost, giving a regret bound that grows like $\sqrt{T}$.

Both algorithms rely on full feedback: once the outcome is revealed, we can score every expert.